阶段三：协同仿真与性能分析

---

1. 协同仿真在做什么

香山 RTL 仿真器（DUT）与 NEMU（参考模型）跑**同一份二进制镜像**。DUT 每提交一条指令，就把架构状态（通用寄存器、向量寄存器、PC）与 NEMU 对应的状态比一次。不一致就停下，打印 PC、指令字、两侧寄存器值。

两个关键点：

- **比对发生在提交时**。乱序执行期间寄存器值随时在变，只有提交后的架构状态才有意义。
- **两侧必须看到同一份内存与 flash**。否则同一个 PC 上执行的指令内容不同，比对毫无意义。

---

2. 一条 `vdot` 怎么在两侧对齐

2.1 前提：两边都得会算

difftest 比的是"同一条指令在两侧算出的架构状态"。DUT（香山 RTL）在阶段二已经认识 `vdot`；NEMU 只认识标准指令，跑到 `0xFA20A1D7` 会报非法或者跑飞。从这一条起两侧分叉，difftest 报的 mismatch 指向的是参考模型不会算，不是设计错误。

所以先要让 NEMU 会算这条指令。

2.2 NEMU 执行一条指令要经过三步

```
取指 → 译码：这条机器码匹配哪个模式？ → 执行：调用对应的函数
                ↑ 需要表项                    ↑ 需要函数，且已登记进名册
```

三处改动正好对应这条链，缺一处链就断：

改哪里	补什么	缺了会怎样
执行函数 `def_EHelper(vdot)`	八项求和的实现、四条非法判据、整寄存器写回	有表项也调不到实现
译码表一行模式	`111110 ? ????? ?????` `010 ????? ????? ??`	认不出这条指令
指令名册 `f(vdot)`	分配执行 ID、进分发表	编译期报错，根本编不过

2.3 `vm=0`：两侧都在译码阶段拦下

模式串第 25 位（`vm`）两侧都写死成 `1`：

```
// RTL
def VDOT_VV = BitPat("b1111101??????????010?????1010111")
```

```
# NEMU（空格只为按操作数分组，便于核对位宽）
def_INSTR_TAB("111110 1 ????? ????? 010 ????? ????? ??", vdot)
```

`funct3` 同样必须写死：`111110 + 110 + 1010111` 是真实存在的 `vwmaccus.vx`，留着通配会把它误认成 `vdot`。

两处都写死后，`vm=0` 的机器码在译码阶段就匹配不到表项，直接落进非法指令，执行函数不会被调用。

3. flash 注入接口

difftest 要把同一份 flash 内容交给两侧。香山侧的代理留了 v1/v2 两个入口，但 NEMU 只导出了 v1——`difftest_load_flash()` 的参数是**文件路径**，拿不到文件就退回 `CONFIG_FLASH_PRESET_CONTENT`；而仿真器手里只有**字节**。两边对不上，表现为"启动阶段就失配"。

解决办法是给 NEMU 补一套 v2 接口：

- `difftest_load_flash_v2()`：接收 DUT 传来的字节，`memcpy` 进 `flash_base`；
- 同时保留 v1 符号，缺 v2 时自动回退。

改动 2 个文件、共 +34 行（`flash.c` +22、`ref.c` +12）。补丁在仓库 `nemu/` 下，可直接 `git apply`。

---

4. 功能验证

类别	用例数	说明
定向	9	正数、负数、边界最小/最大、全零、正负混合、`vl=8/4/0`
非法	3	`vm=0`、`vsew=e16`、`vlmul=m2`
随机	100	随机 int8，固定种子，软件独立计算期望值

合计 **112 次执行**（9 + 3 + 100）。

非法用例只在 RTL 侧观察 `mcause`。原因是这套 difftest 在指令触发 trap 时两侧不同步：DUT 提交带异常的那条指令时，参考模型侧的 `mcause`/`mepc` 仍是旧值，比对当场 abort。用两条与 `vdot` 无关的对照实验可以确认这是共性问题——`ecall` 失配（`mcause=0xb` 对 `0`）、全零指令失配（`mcause=2`

对 `0`），与 `vm=0` 的表现完全一样。所以异常类用例（非法条件、`vstart≠0`）都以 RTL 侧的 `mcause` 为准。对照实验的原始输出在 `evidence/final8/trap-diff-control.txt`。

期望值由独立的 C 实现产生，不用手算。随机用例用固定种子的 xorshift32，保证可复现。

---

5. 性能：两组数字怎么来的

5.1 测法

两组数字都来自 `tests/baseline/perf_latency.c`：同一份数据（`vs1 = vs2 = 1..8`）、同样循环 100 次（`NITER=100`），只改变**计时区间**与**循环体的构成**。

单次执行

两边都把 `vsetvli` 和循环开销算进去。

vdot 的循环体：

```
vsetvli   t1, t0, e8, m1, ta, ma     # t0=8 在循环外设好；只需 vtype，vdot 不看 vl
vdot.vv   v3, v1, v2                 # 一条指令完成全部工作
```

> VL 值不影响 `vdot.vv`。

baseline 的循环体：

```
li          t0, 8                    # VL = 8
vsetvli     t1, t0, e8, m1, ta, ma   # ① 切到 8 位元素
vwmul.vv    v4, v1, v2               # ② 逐元素乘，结果加宽成 16 位
vsetvli     t1, t0, e16, m1, ta, ma  # ③ 切到 16 位元素
vwredsum.vs v3, v4, v0               # ④ 16 位元素归约求和
```

baseline 一轮 **5 条**，vdot 一轮 **3 条**。实测 **5 拍 vs 3 拍 = 1.67×**。

> 末尾不再还原 vtype：回到 ① 时那一句本来就会重设 e8/m1，再设一次是多余的一条指令。

连续运算

不做循环，把 100 条指令直接展开，测的是**发射间隔**。

vdot 侧： `vsetvli` 只做一次。

```
vsetvli   t1, t0, e8, m1, ta, ma     # 只做一次
.rept 100
  vdot.vv v3, v1, v2                 # 100 条连续运算
.endr
```

baseline 侧：展开它那四条不可分割的序列。

```
.rept 100
  vsetvli     t1, t0, e8, m1, ta, ma
  vwmul.vv    v4, v1, v2
  vsetvli     t1, t0, e16, m1, ta, ma
  vwredsum.vs v3, v4, v0
.endr
```

实测 **17 拍 vs 1 拍**。1 拍/条说明流水线每拍能吃进一条 `vdot`，这是硬件的发射上限。

5.2 两组数字

口径	baseline	vdot	比值	反映什么
单次执行	5 拍	3 拍	**1.67×**	同样任务、同样循环下的平均周期
连续运算	17 拍	1 拍	**17.0×**	不中断地连续发射时的间隔

延迟另有单独一组数，在 `evidence/final8/perf.log` 的 `[A]` 行：baseline 11 拍、vdot 1 拍。它含两次 `csrr` 的测量开销，绝对值偏大，只用来看差距量级。它与 §8 的 2 拍不是一回事：2 拍是 FU 边界的流水级数，`[A]` 是软件夹在指令两端的墙钟差。

6. 结构量化

环境里没有 PDK 与标准单元库，做不了真实面积。替代做法是 Yosys 通用门级综合：工艺无关的门级网表，统计等效门数与最长组合路径。

`VDotUnit`：**4,887 等效门**，最长组合路径（ltp）**35 级**。

6.1 量级换算

单个 8×8 有符号乘法器（`mul8s.sv`，同一流程）是 **391 门**：

```
4,887 ÷ 391 ≈ 12.5
```

整个单元的成本约等于 **12.5 个裸的 8×8 乘法器**。

6.2 为什么不做构成分解

按「8 个乘法器 + 7 个加法器」逐个综合再相加，得到的数不可用，原因有两层：

- 仓库里三个乘法器文件（`mul8s.sv`、`mul16s.sv`、`mul16s_trunc.sv`）的输入端口都只有 8 位，符号扩展到 16 位这一步 Yosys 看得穿，同一流程下三者都综合成 **391 门**，测不出 16×16 的真实单价；
- 即便测得出，ABC 在整单元综合时会做跨实例优化（部分积的公共子表达式、加法树进位链重排），单件最优不等于合起来最优。

因此这里只给总量与量级换算，不做百分比归因。

7. 扩展性

`VDotUnit` 的元素数 `NELEM` 是参数，不是硬编码的 8。把综合模型按不同配置各跑一遍：

NELEM	VLEN	门数	树层	ltp
4	64	2,394	2	31
8	128	4,887	3	35
8	256	4,887	3	35
16	256	9,894	4	40
32	256	19,851	5	44

三件事：

1. **元素数近似线性**：8→16 元素门数 2.02×，树深 +1 级，ltp 35→40（约 626 门/元素，即一个乘法器加上树上摊销的加法器）；
2. **VLEN 单独加宽不花门**：`NELEM` 不变时，VLEN 128→256 的门数与 ltp 完全相同——多出来的 lane 只是端口变宽，不进逻辑。这一条只对数据通路成立，整机改 VLEN 要动平台侧。
3. **扩展形态跑过功能自检**：`NELEM=16 / VLEN=256` 的模型在 Verilator 上做了 850 次位精确比对，0 失败；`NELEM=8` 同构配置 650 次，0 失败。

语义上也不依赖 8：乘法阵列与归约树都按 `NELEM` 展开，改元素数只需改这一个常数并同步参考模型；改成 16 元素的形式，`vd[31:0]` 的 32 位回绕就不再成立（16 个 int8 乘积最大 2^19），需要同时改累加位宽。树深增加后是否插流水寄存器取决于目标频率，本作品没有 STA，所以只给深度、不下结论。

证据：`evidence/synth/nelem-scaling.txt`、`width-comparison.txt`、`reducer-comparison.txt`。

8. 波形

功能对不对靠 difftest 比，时序对不对只能看波形。用带 `--trace` 的仿真器重抓了一次：

```
# 带 trace 的仿真器单独构建，构建完把 build/emu 另存一份（下文记作 $EMU_TRACE，路径自定）
make emu EMU_TRACE=1
EMU_TRACE=$HOME/emu.trace2

"$EMU_TRACE" -i vdot_wave.bin --no-diff -b 8300 -e 8340 -C 8340 \
    --dump-wave --wave-path=/tmp/vdot_win.vcd
```

被测程序只跑一条 `vdot.vv`（`tests/vdot/vdot_wave.c`，`vs1 = vs2 = 0x0101…01`，期望 `0x00000008`）。窗口取 8300–8340 拍，原始 VCD 79 MB，压缩后 8.0 MB，归档在 `evidence/wave/vdot-8318-window.vcd.gz`。

按拍读 `VIMacU` 的输入输出（`rtl/wave_vimac_cycle.py`）：

拍	in_valid	out_valid	res_data	pdest	fuOpType
8317	0	0	—	—	上一条的残留
8318	**1**	0	0x00000000	—	**0b001100111 = 0x067（vdot）**
8319	0	0	0x00000000	—	0x067
8320	0	**1**	**0x00000008**	0b00110011 = 51（v3）	0x067

三件事由此确认：

1. **FU 内 2 拍**：输入在 8318、输出在 8320，跟 `VimacCfg.latency = CertainLatency(2)`（`FuConfig.scala:559`）对得上，不是我们另设的延迟；
2. **结果是组合算出来的**：`VDotUnit` 里没有寄存器，8318 那拍把 8 个 int8 乘积加完，8320 拍出来的 `0x00000008` 就是 8 个 1 的和；
3. **写回目标正确**：`pdest=51` 是 `v3`，与指令里的 `vd` 一致。

---

9. 交付物清单

类别	内容
参考模型补丁	`nemu/difftest-flash-v2.patch`（2 文件 +34 行）、`nemu/vdot.patch`（3 文件 +37 行）
测试程序	`tests/vdot/vdot_basic_tests.c`、`vdot_illegal_tests.c`、`vdot_random_tests.c`
证据	`evidence/final8/`（定版构建的 112 次执行日志）、`evidence/a8/`（性能）、`evidence/synth/`（综合）、`evidence/wave/`（波形窗口与逐拍记录）
复现脚本	`scripts/repro-stage3.sh`（一键复现）、`scripts/run-all.sh`（回归本体）

10. 复现

```
bash scripts/repro-stage3.sh          # 快速：difftest 9 例 + 非法 4 例 + vstart 3 例
bash scripts/repro-stage3.sh full     # 全量：再加 100 组随机对拍与性能
```

它调 `scripts/run-all.sh`，把每一项的原始日志写进 `evidence/regress/`，最后打印汇总表。

判定看日志，每项要出现 `RESULT: PASS`，且里面没有 `different at`。